FSM

↳ sequential clocked circuits

@ Analysis of FSM

Step 1: for $A(t+1) = x \,\&\, (A(t) \,\hat{}\, B(t))$

$B(t+1) = \bar{x} \,|\, (\bar{B}(t) \,\&\, A(t))$

$Y = AB$

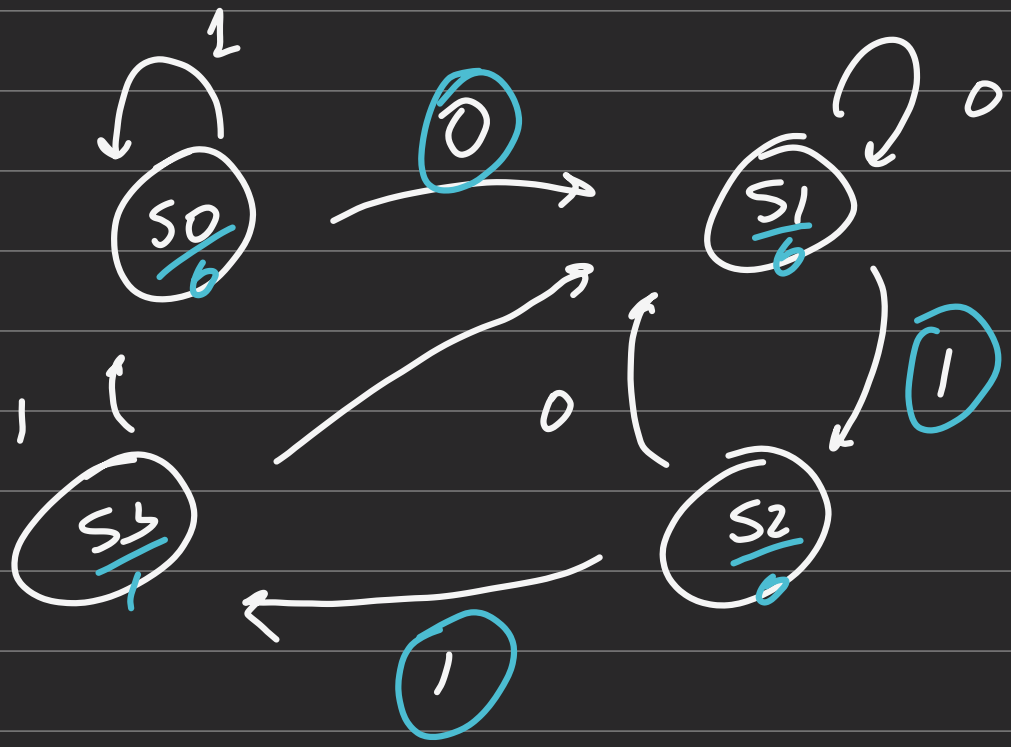
Analyse the function

Step 2: draw state table

	current state			Next state		output
	A	B	X	A	B	Y
S0	0	0	0	0	1	
	0	0	1	0	0	
S1	0	1	0	0	1	
	0	1	1	1	0	
S2						

S3

step 3: draw state diagram



moore machine: output does not depends on input

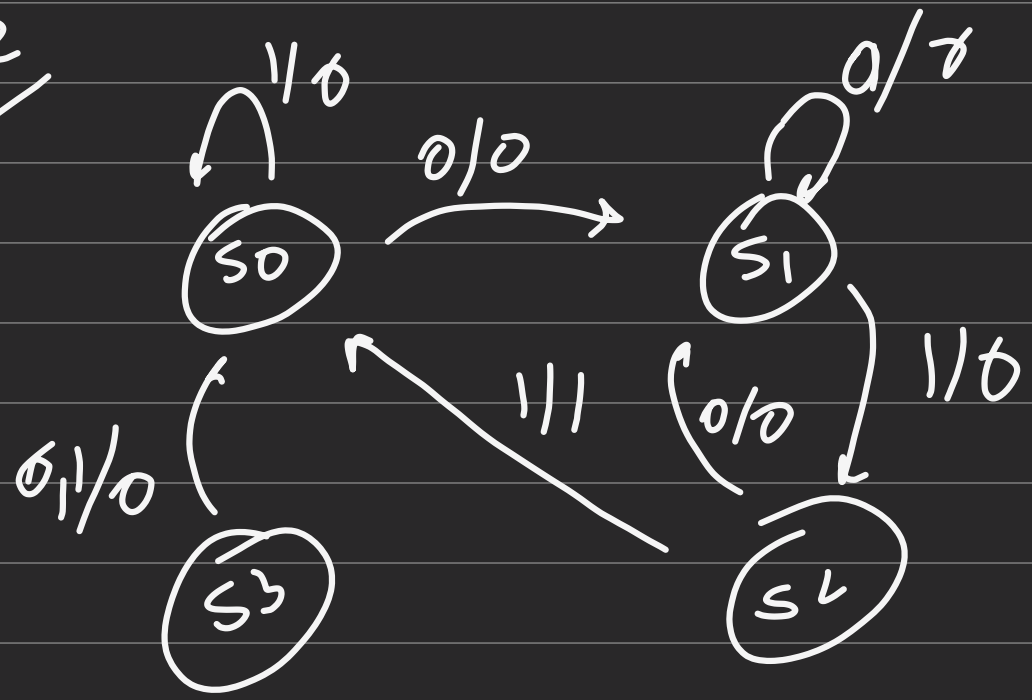
Step 4: pattern detection

011 → y = 1

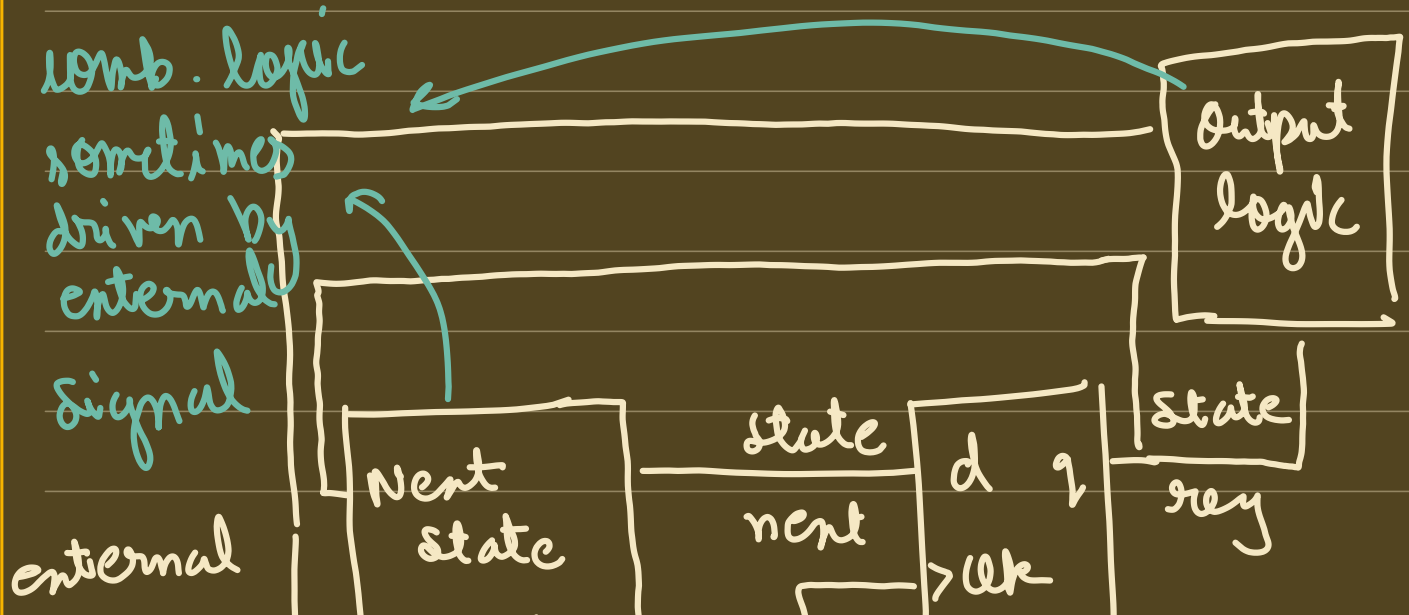
VS

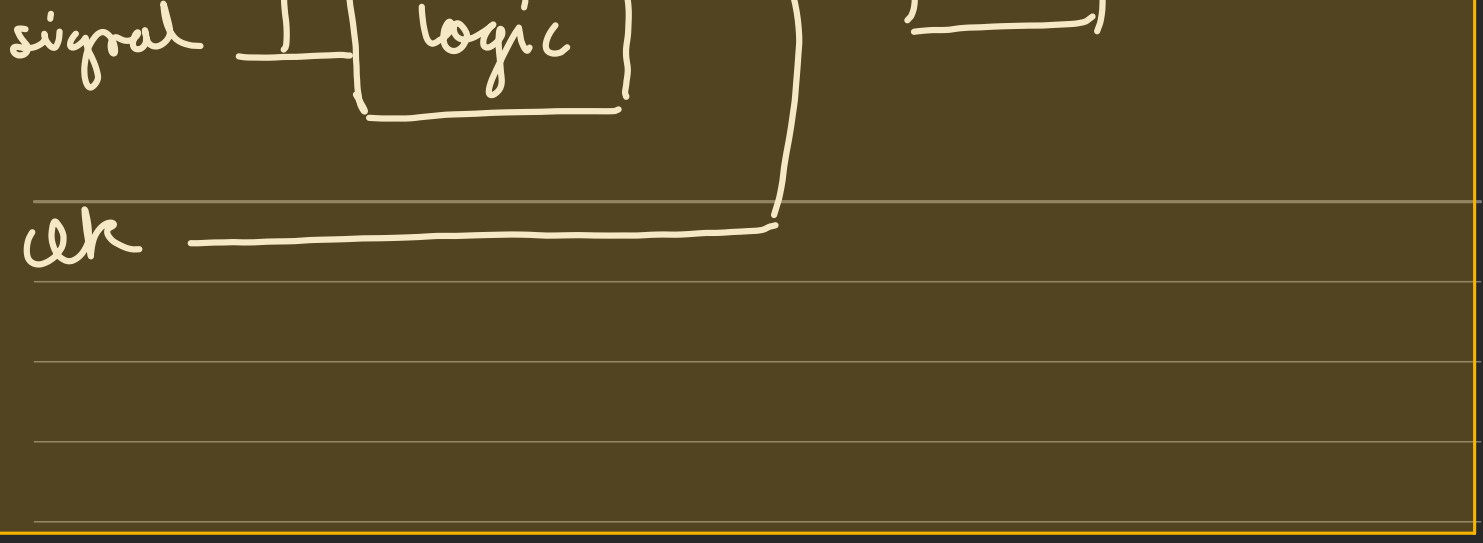
mealy machine : output is fun of inputs well

i.e



General Synchronous system

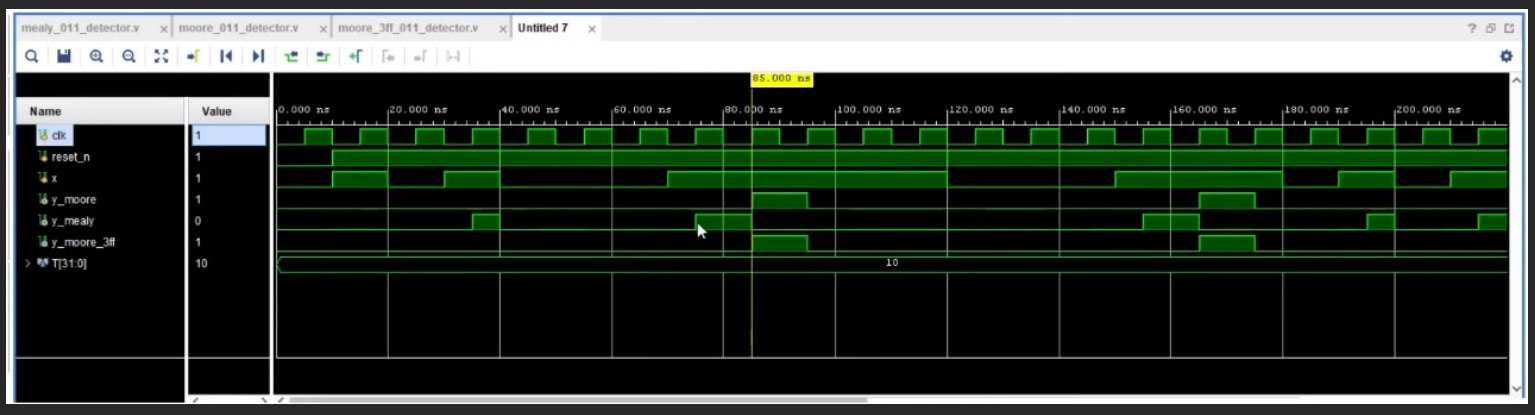




so mealy machine is not nessarily synchronous, it can change whenever input changes

↳ differene in terms of output

mealy usually responds faster



→ 011 sequence detator

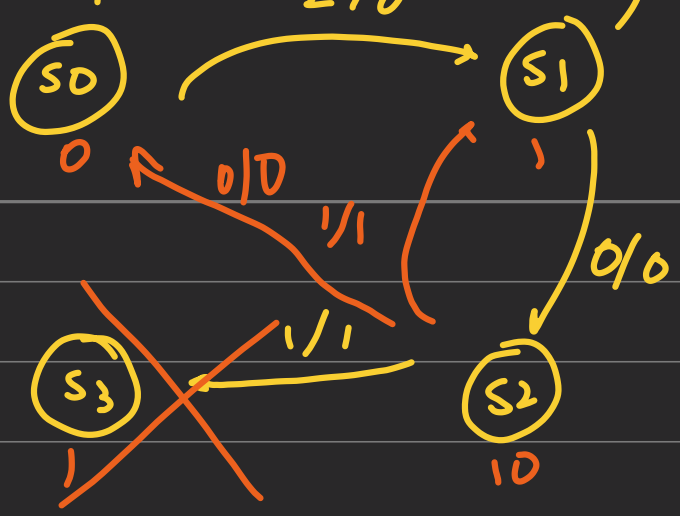
② Design of FSM

- Draw the state diagram based on specifications & desired behaviour.
- Reduce no. of states if necessary.
- Assign binary values to states.
- Obtain the binary-coded state table.
- Choose FF type.
- Get simplified FF input equation & output eqn.
- Draw circuit diagram.

↳ Overlap → moving window

overlap not allowed → windowed window

i.e 101 → overlapping allowed



assigning binary values :

S0 → 00
S1 → 01
S2 → 10

} 2 FF req.

OR

one hot

S0 → 001
S1 → 010
S2 → 100

} 3 FF used

but next state logic will be easier

Present state	Next state		Output	
	x=0	x=1	x=0	x=1
A B	A B	A B	y	y
0 0	0 0	0 1	0	0
0 1	1 0	0 1	0	0
1 0	0 0	0 1	0	1
1 1	x x	x x	x	x

AB ↙ A

x \ AB	00	01	11	10
0	0	1	X	0
1	0	0	X	0

x \ AB	00	01	11	10
0	0	0	X	0
1	1	1	X	1

← B

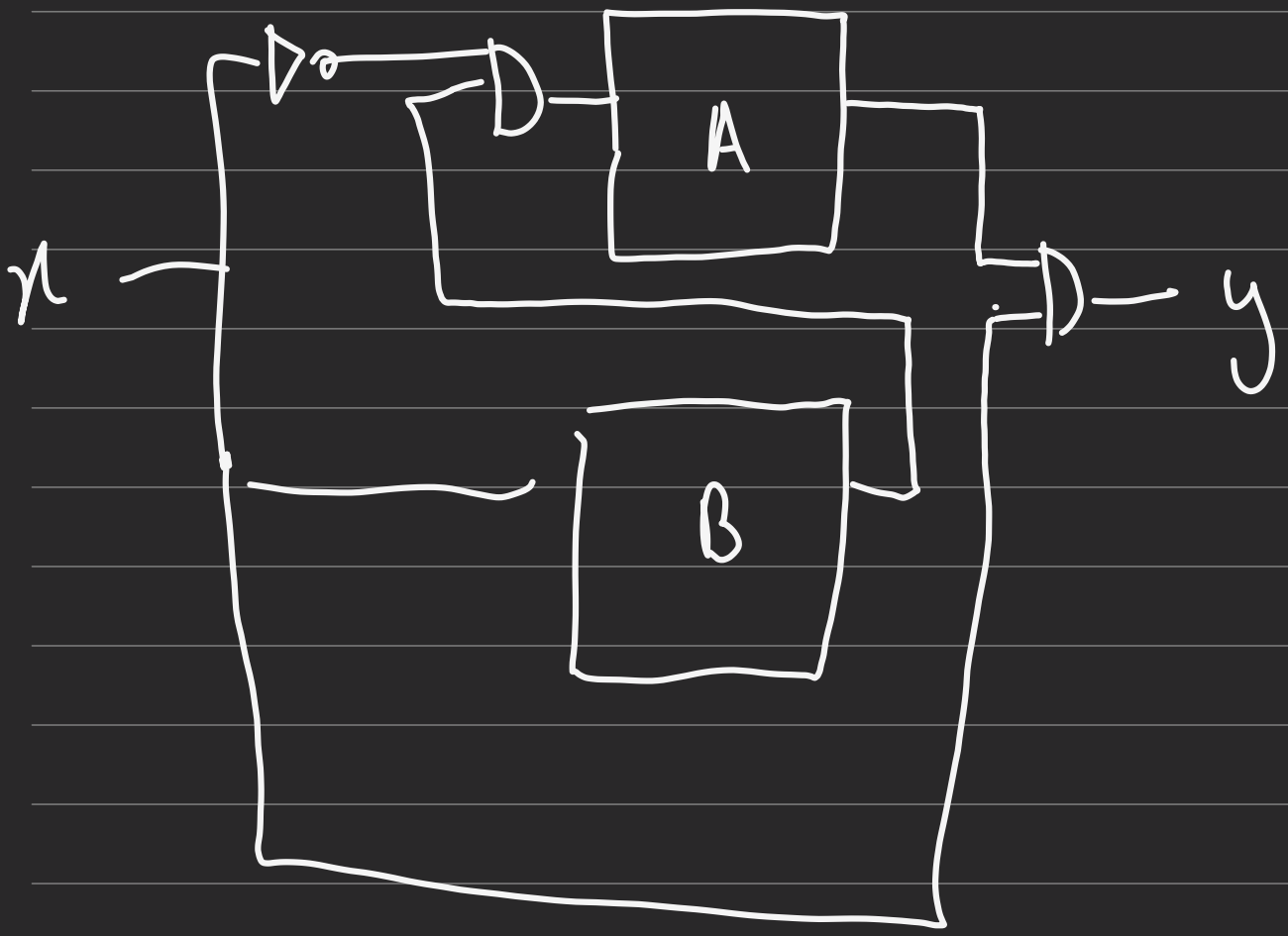
x \ AB	00	01	11	10
0	0	0	X	0
1	0	0	X	1

← y

$A(t+1) = \bar{x}B$

$B(t+1) = x$

$y(t+1) = x A$



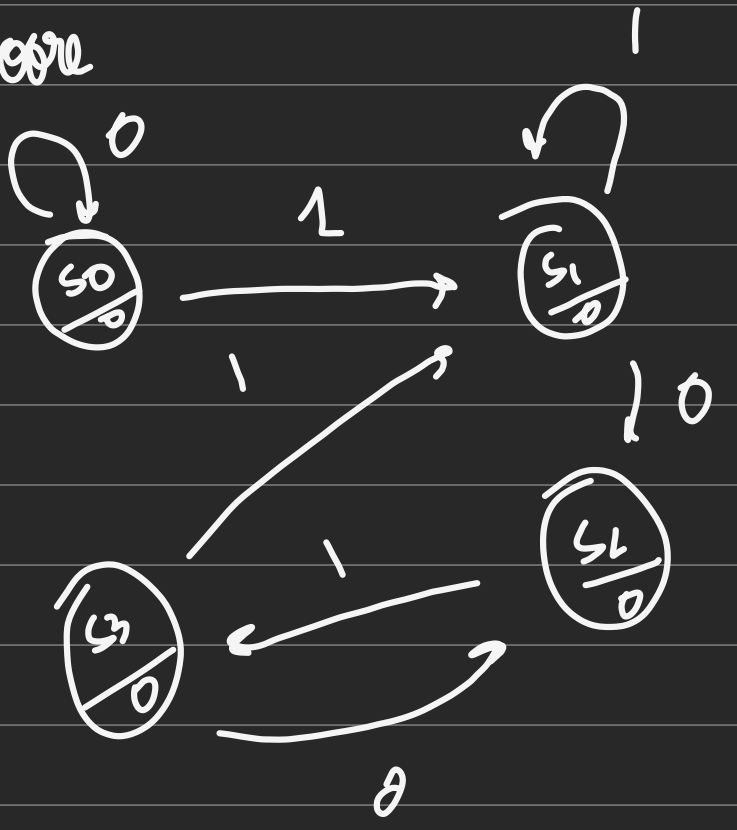
↳ 1 more diff. b/w mealy & moore.

· moore gives output in next state

	x	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0	
Moore	y		0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0
	Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

	x	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0
Mealy	y	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0
	Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

↳ Moore



S0 : 0
S1 : 1
S2 : 10
S3 : 101

③ Verilog Description of FSM

module moore_101_detector (

_____

_____

_____

_____);

reg [1:0] state, reg state_next;

// state register

```
always @(posedge clk, negedge
          reset_n)
begin
    if (~reset_n)
        state_reg <= S0;
    else
        state_reg <= next
                     state;
end
```

// next_state logic

```
always @(*)
begin
    case (state_reg)
      S0: if (x)
            state next = S1;
          else
            state next = S0;
      S1: ____
          ____
      S2: ____
          ____
      S3: ____
          ____
```

default : state = next_state;

endcase.

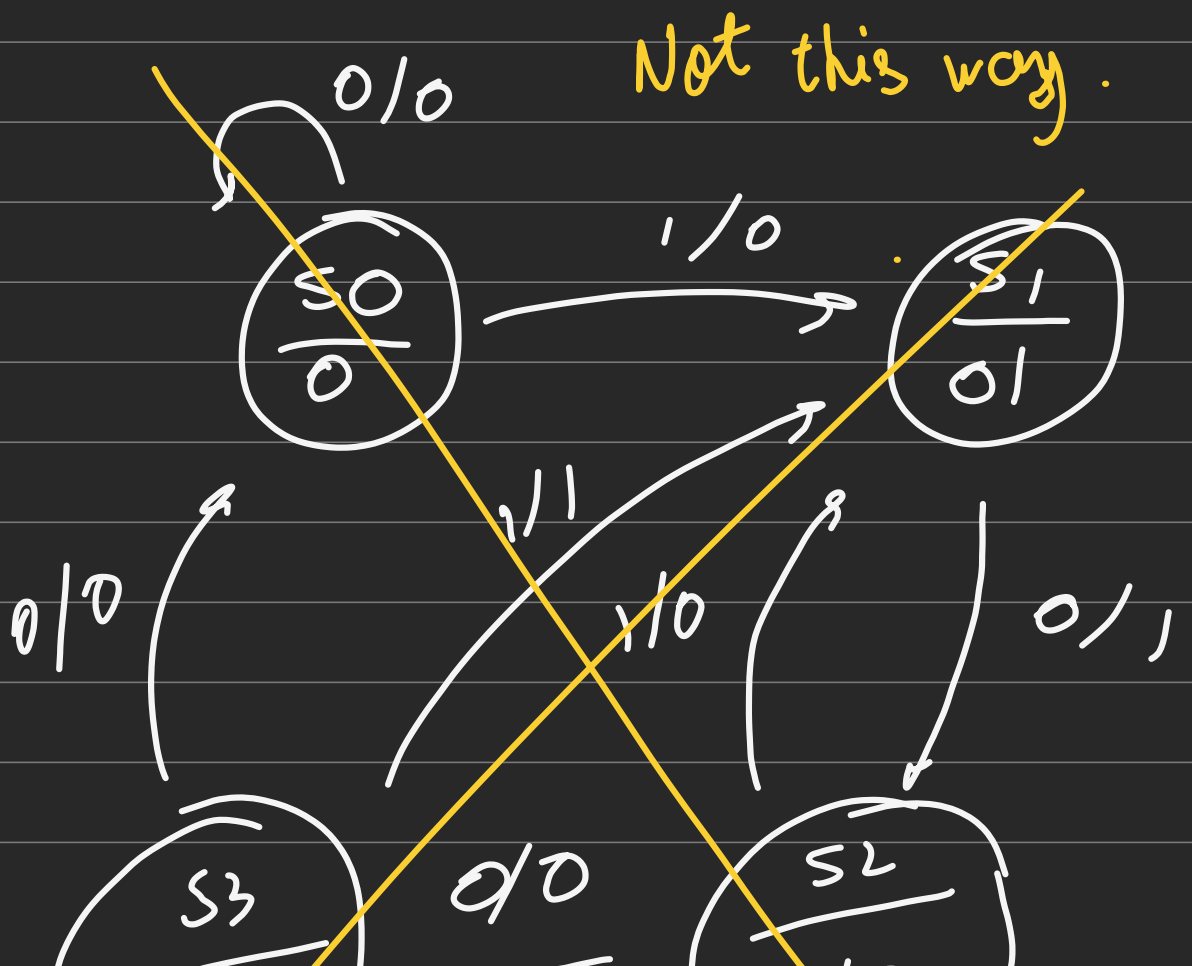
end.

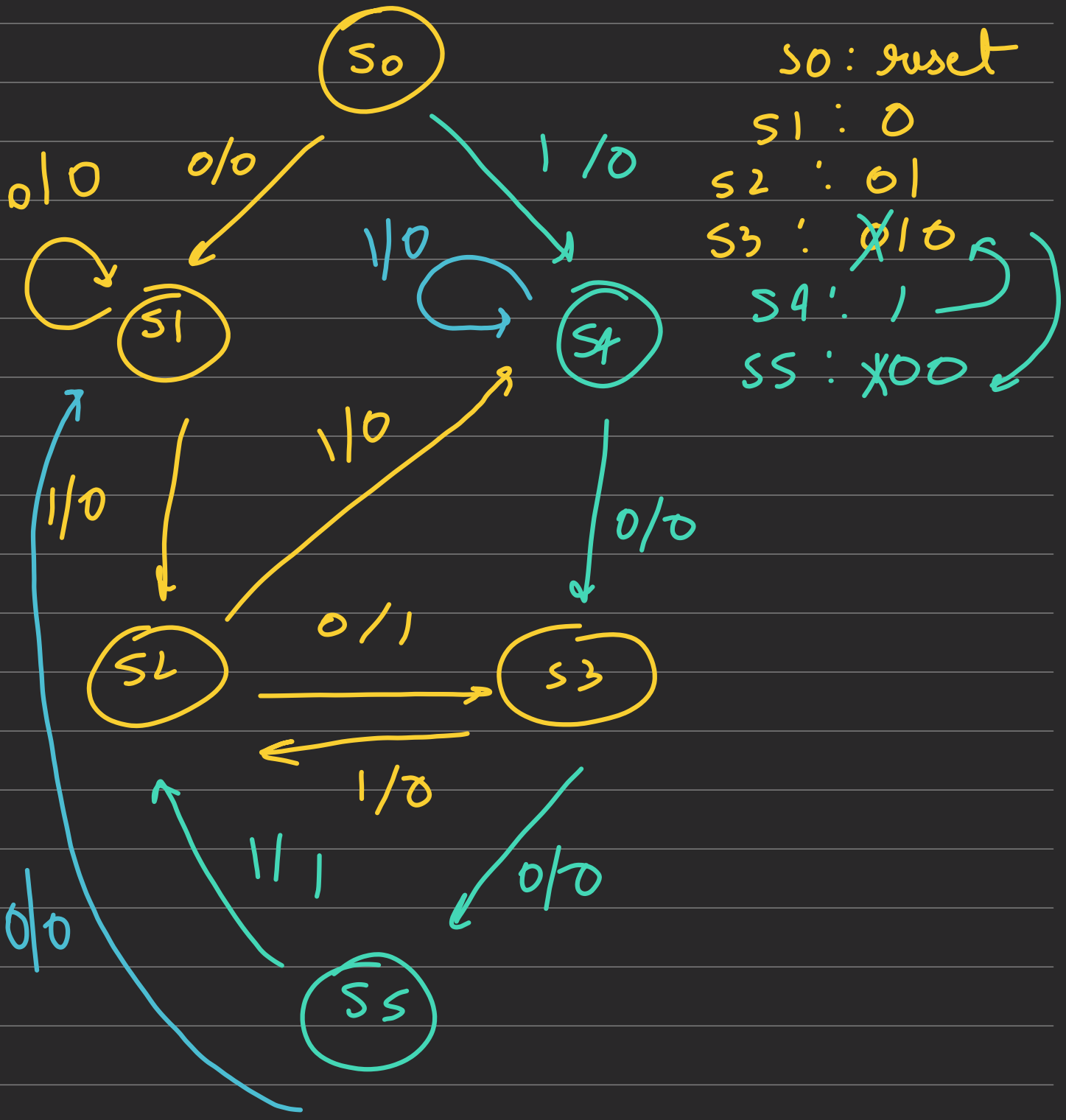
// output logic

assign y = (state_reg == S3)

Complex FSM

Ex:1 010
1001 } detect ; Mealy





5 states → 3 f.f

+x

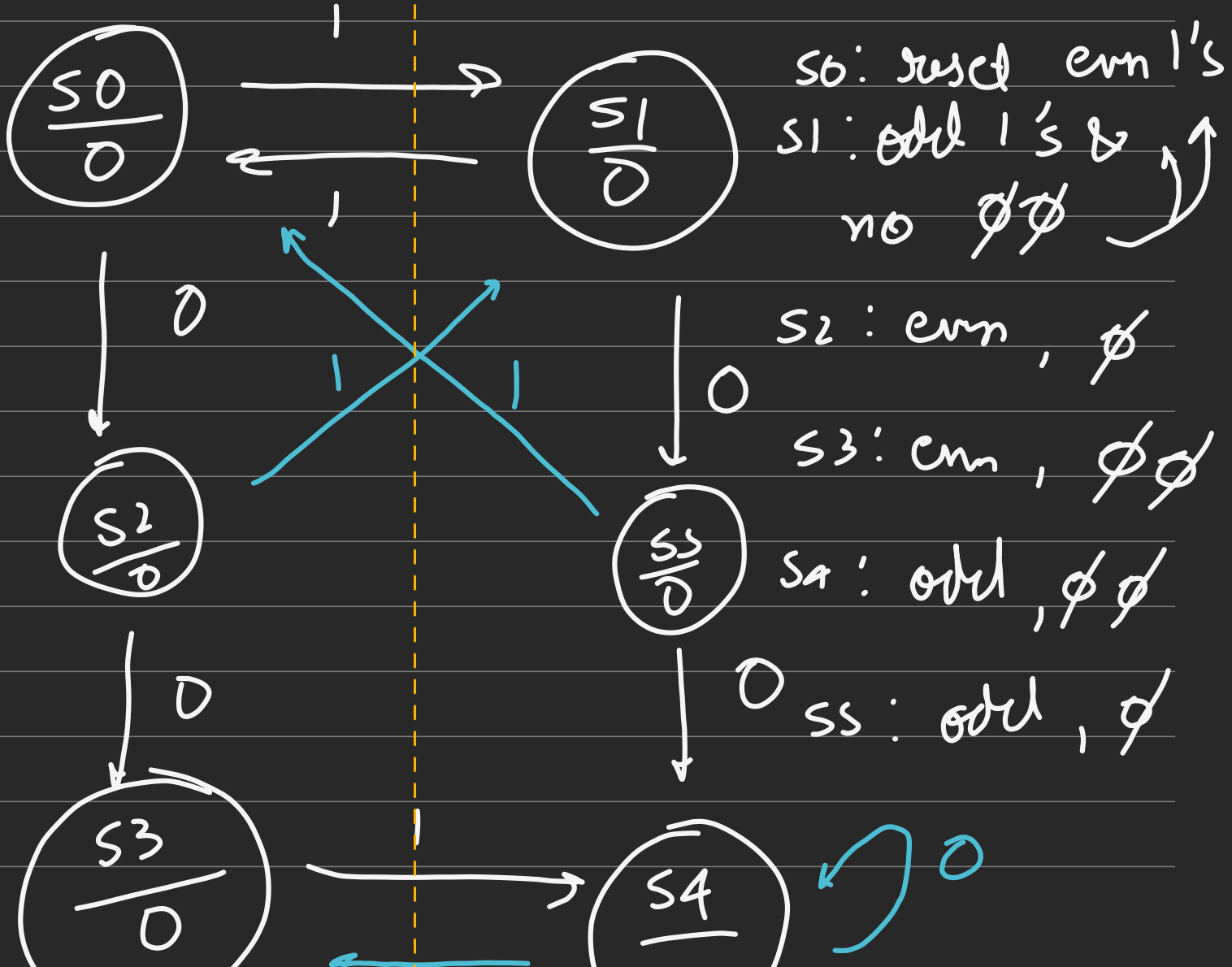
4 variable K Map

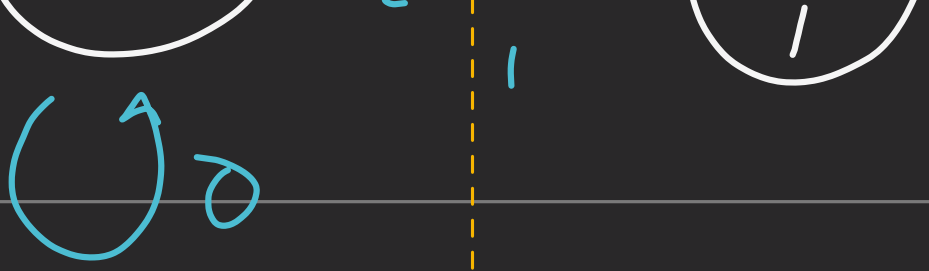
Ex: 2

gives 1 after odd no. of 1
0 — even no — 1

& starts wd this after two 0's.

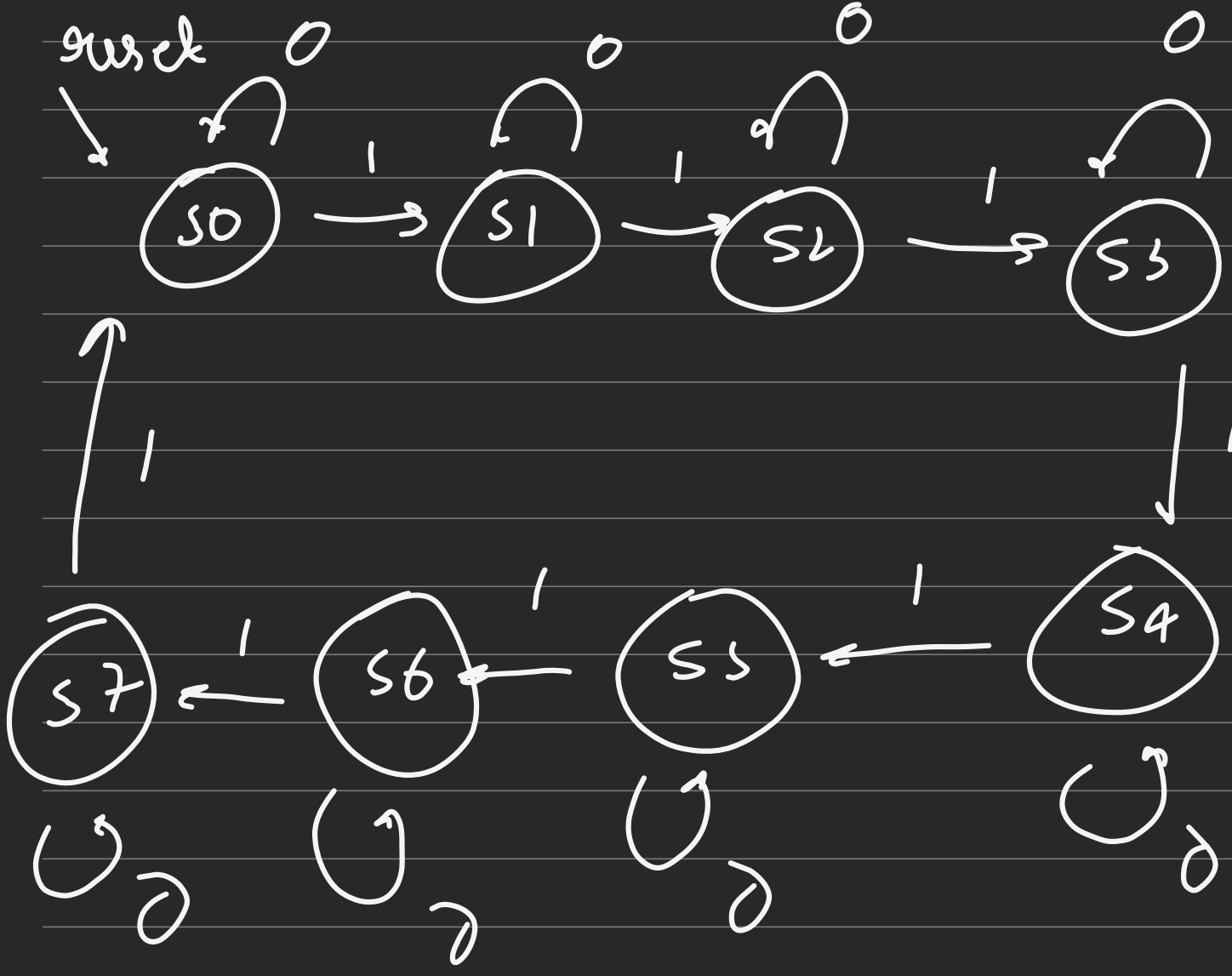
→ moore





even side odd side

Counter as FSM



↳ ② localparam

↳ cannot be modified once after declaration

```
localparam S0 = 0, S1 = 1, S2 = 2,
           S3 = 3;
```

② parameter

↳ configurable const. used during module instantiation.

```
module simple_reg
#(parameter N = 8) (
    ...
    ...
    ...
);
```

③ integer

↳ 32 bit signed variable.

↳ used for non-hardware purpose i.e loop counters or temporary calculation.

④ genvar

↳ compile time - variable

↳ used inside generate loop.